FIAP ON

MAKING STRATEGIC DECISIONS IN THE CLOUD

ARQUITETURAS SERVERLESS E DESACOPLADDAS

10

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Utilização ao longo do tempo 4
Figura 2 – *Serverless:* mudança de paradigma 5
Figura 3 – AWS Lambda – Serverless Function 6
Figura 4 – Processamento de LOG 9
Figura 6 - Portal Azure - Function App 10
Figura 7 - Create Function 11
Figura 8 - Configurações Básicas 12
Figura 9 - Hosting - modo de consumo da Function App 13
Figura 10 – Storage 13
Figura 11 - Networking (rede) 14
Figura 12 – Monitoring 14
Figura 13 - Deployment 15
Figura 14 – Configuração de TAGs 16
Figura 15 - Revisar e criar 16
Figura 16 - Disponibilização da Function App 17
Figura 17 - Criar uma função 18
Figura 18 - Criação de uma função HTTP Trigger 19
Figura 19 - Configuração da função HTTP Trigger 20
Figura 20 - Código gerado automaticamente 20
Figura 21 - URL da função gerada. 21
Figura 22 - Teste da função 21
Figura 23 - Teste da função 22

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 APLICAÇÕES EM NUVEM MAIS ESCALÁVEIS E COM MENOR CUSTO.....4
1.1 Introdução 4
1.2 Cronologia.....5
1.3 Vantagens e desvantagens.....7
1.4 Arquitetura desacoplada.....8
2 HANDS-ON – IMPLEMENTAÇÃO DE UMA FUNÇÃO SERVERLESS.....10
ATIVIDADE TREINO.....23
REFERÊNCIAS.....24

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 APLICAÇÕES EM NUVEM MAIS ESCALÁVEIS E COM MENOR CUSTO

1.1 Introdução

A arquitetura *serverless*, também conhecida como *serverless computing*, é um paradigma de desenvolvimento, projeto e entrega de aplicações orientadas a eventos. Tem como principal objetivo permitir que as empresas de TI e seus desenvolvedores criem e mantenham as aplicações sem se preocuparem com infraestrutura subjacente. Ou seja, os desenvolvedores não precisam instalar, configurar e manter os recursos como servidores, sistemas operacionais e serviços.

Apesar do nome sugestivo, é claro que as aplicações desenvolvidas na arquitetura *serverless* usam processamento e armazenamento de servidores para executarem sua lógica. Na realidade, o termo *serverless* descreve a experiência do usuário (desenvolvedor) com esses servidores, que se tornam transparentes, pois não necessitam que sejam gerenciados ou configurados por eles.

O conceito surgiu por volta do ano de 2004 (DOMALIK, 2019), mas, conforme o mesmo artigo, só ganhou força de utilização (interesse) por volta de 2014, com o lançamento do AWS Lambda, a tecnologia *serverless* da nuvem Amazon. A Figura 1“Utilização ao longo do tempo” apresenta a utilização do *Serverless Computing* ao longo do tempo, é possível notar um expressivo aumento por volta de 2014.

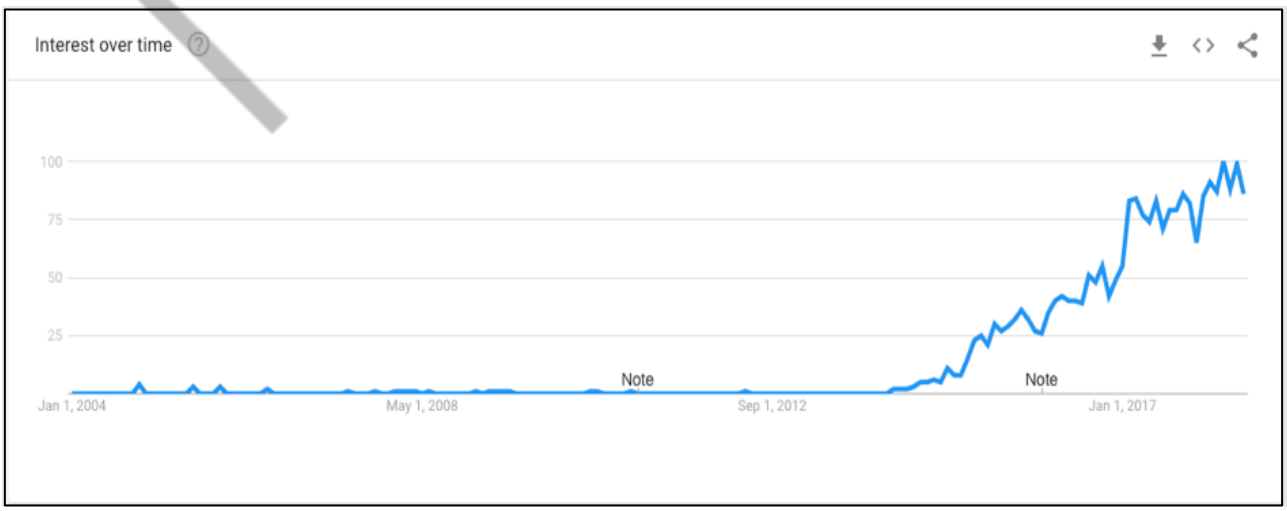

Figura 1 – Utilização ao longo do tempo
Fonte: IBM (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.2 Cronologia

Para compreender melhor a arquitetura *serverless* e como surgiu sua necessidade, é importante analisar os paradigmas e arquiteturas que a precederam.

A Figura "*Serverless:* mudança de paradigma" ilustra muito bem a mudança de paradigma na utilização da tecnologia *serverless*. O eixo Y (vertical) apresenta o crescimento do foco na lógica de negócio ao se utilizar o desenvolvimento com *serverless*. O eixo X (horizontal) apresenta a redução de controle e manutenção na infraestrutura por parte dos desenvolvedores.

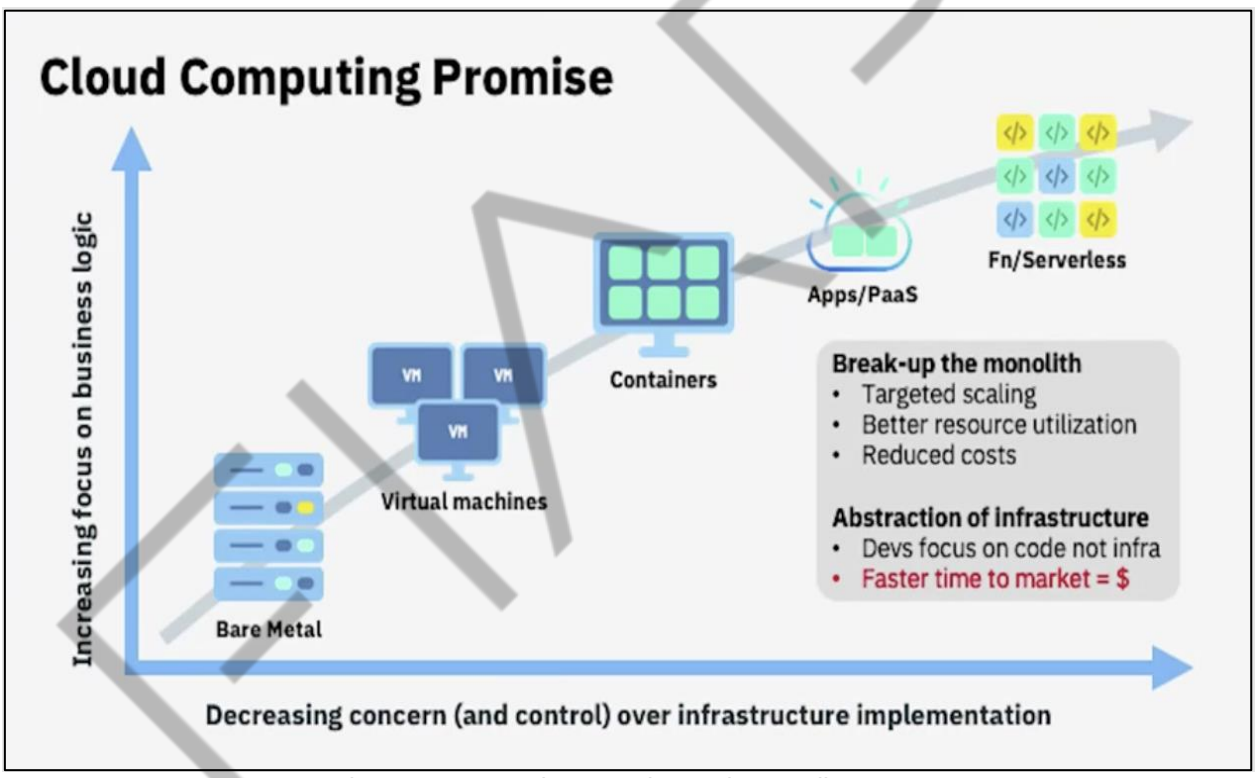

Figura 2 – *Serverless:* mudança de paradigma
Fonte: IBM (2021)

No paradigma "*Bare Metal*", o provedor de nuvem é responsável por oferecer um espaço físico (energia e segurança) aos servidores. A maioria das responsabilidades fica com a empresa contratante, que deve se preocupar com a instalação, configuração, manutenção e o gerenciamento das máquinas. Além disso, não pode contar com a escalabilidade necessária para os dias atuais.

Passando para o paradigma de máquinas virtuais, a empresa contratante ganha em escalabilidade, ou seja, à medida que seus negócios crescem (ou

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

diminuem), pode alocar ou desalocar recursos computacionais, mas ainda continua com a responsabilidade de instalação, configuração, manutenção e gerenciamento das máquinas.

No modelo de containers, a maior parte de infraestrutura fica sob responsabilidade do provedor de nuvem. Os desenvolvedores ou a empresa contratante são responsáveis por configurar somente a aplicação e os recursos mínimos necessários para a aplicação funcionar.

Por fim, a computação *serverless* permite que os desenvolvedores se mantenham focados na lógica de negócio a ser implementada, permitindo abstraírem-se de toda a preocupação com a infraestrutura.

A Figura “AWS Lambda – Serverless Function” apresenta um exemplo para disponibilizar um serviço ou uma aplicação que proveem informações do tempo para os usuários. Da esquerda para a direita, são mostrados os componentes da aplicação, começando com o armazenamento do código front-end no serviço Amazon S3 (Simple Storage Service).

Quando o usuário clica na função requisitando as informações do tempo, a solicitação é direcionada para uma API Gateway (Application Program Interface Gateway) que serve para controlar e redirecionar as chamadas para os programas (inclusive as funções serverless). Assim, a função serverless (AWS Lambda) desenvolvida e disponibilizada é acionada e consulta um banco de dados DynomoDB com as informações do tempo.

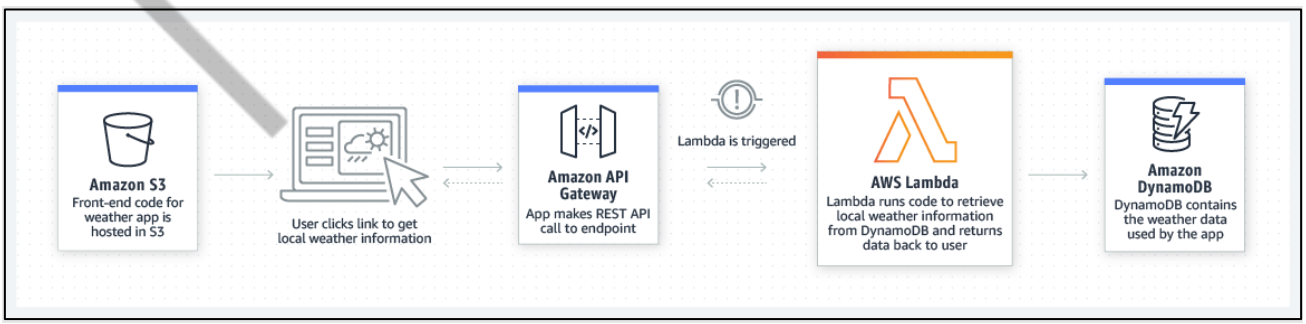

Figura 3 – AWS Lambda – Serverless Function
Fonte: AWS (2022)

Nesse cenário, os desenvolvedores se preocuparam exclusivamente com o desenvolvimento da função serverless e posterior entrega (deploy). Não é necessária a definição da arquitetura física de servidores e ainda pode contar com a escalabilidade automática, caso a aplicação se torne muito acessada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Vantagens e desvantagens

O modelo sem servidor (*serverless*), como todo novo paradigma, traz algumas vantagens (Cloudflare) e também algumas desvantagens em relação aos modelos anteriores.

Vantagens:

- **Custos mais baixos:** o modelo de cobrança é o *pay-as-you-go*, ou seja, é relativamente proporcional ao consumo (execução) da aplicação (função). Não é necessário contratar e alocar previamente recursos de processamento e memória que podem se tornar subutilizados. Para se ter uma ideia, o AWS lambda custa US$ 0,20 a cada milhão de requisições para cada função disponibilizada.
- **Escalabilidade:** na maioria dos provedores de nuvem, não é necessária a preocupação com a disponibilidade nem a escalabilidade. São recursos inerentes ao modelo *serverless*. Ou seja, quanto mais requisições, mais recursos computacionais são fornecidos de forma automática.
- **Códigos reduzidos:** os códigos desenvolvidos são focados em soluções simples para executar uma única função.
- **Retorno mais rápido:** a codificação *serverless* pode diminuir significativamente o tempo de lançamento no mercado. Em vez de o time se preocupar com questões de infraestrutura, ele pode focar na solução de forma inovadora e otimizada.

Desvantagens:

- **Desempenho:** com as funções *serverless,* pode ocorrer o cold-start. Ou seja, a função não está inicializada, o que significa que ela deve ser lida e colocada em memória principal para começar a ser executada. Isso pode demorar algum tempo, que é imperceptível para algumas aplicações, mas pode ser inaceitável para outras, por exemplo, as aplicações para operações financeiras.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Dependência do provedor de nuvem:** apesar da codificação ser independente do provedor, a arquitetura e as entregas (deploys) podem se tornar muito dependentes dos provedores à medida que utilizam mais as particularidades de cada ecossistema.
- **Testes:** os testes integrados são mais complexos de serem realizados, pois isso significa invariavelmente efetuar testes distribuídos, chamando cada função *serverless*.

1.4 Arquitetura desacoplada

A arquitetura desacoplada é um tipo de arquitetura de computação que permite que componentes ou partes do sistema sejam executados independentemente, porém ainda fazem interface uns com os outros.

Os sistemas mais antigos eram monolíticos, ou seja, era um único bloco de programa que abrigava todas as funcionalidades do sistema, incluindo a interface com o usuário, o acesso aos dados e as respectivas funções. Esses sistemas, ao serem iniciados, funcionam como único executável na máquina, ou seja, um único processo alocado em memória e concorrendo ao processador.

As desvantagens dessa arquitetura são inúmeras, entre elas, o alto acoplamento entre as funcionalidades, o que causa uma dependência muito grande entre todas as funções do sistema.

A execução monolítica impede, muitas vezes, o processamento paralelo, que poderia usufruir dos múltiplos processadores de uma máquina. A manutenção no código altamente acoplado pode gerar erros graves em outras partes que não foram alteradas. A reutilização de código é muito baixa. Entre muitas outras desvantagens.

Ao longo do tempo, os sistemas foram sendo desenvolvidos em uma arquitetura mais modular, na qual algumas camadas do sistema foram desacopladas e funcionam de forma independente. Por exemplo, a camada de interface com o usuário, a camada de acesso aos dados (banco de dados) e a camada de aplicação foram desacopladas, permitindo, inclusive, serem executadas em servidores diferentes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A evolução da tecnologia e do desenvolvimento procura cada vez mais aumentar o desacoplamento das aplicações. Assim, torna-se possível construir sistemas grandes que podem ser executados por diversos serviços individuais, porém ainda integrados entre si, com o benefício de poderem ter processamento paralelo, execução independente, manutenções mais simples e rápidas, entre muitos outros benefícios.

Os serviços e recursos disponibilizados no ambiente em nuvem vão ao encontro dessa arquitetura desacoplada. Cada parte da aplicação pode ser desenvolvida ou configurada em um serviço específico, inclusive em diferentes nuvens (multicloud).

Inclusive a utilização de funções *serverless* vem sendo muito mais explorada. Sistemas grandes estão sendo desenvolvidos com muitos recursos em nuvem, especialmente as funções *serverless*.

A Figura “Processamento de LOG” apresenta mais um exemplo de aplicação com utilização de recursos na nuvem e arquitetura serverless. É possível observar que cada recurso (Amazon CloudWatch Logs, AWS Lambda, Amazon DynamoDB, Amazon S3, Amazon Redshift) é individual e tem suas configurações e desenvolvimentos independentes uns dos outros, mas mantém a integração.

O exemplo da Figura “Processamento de LOG” mostra a captura de arquivos de LOGs dos mais variados recursos (aplicação, banco de dados, rede etc.) e posterior processamento por uma função *serverless,* e depois o armazenamento e a disponibilização em diferentes locais e formatos (DynamoDB, S3 e Redshift).

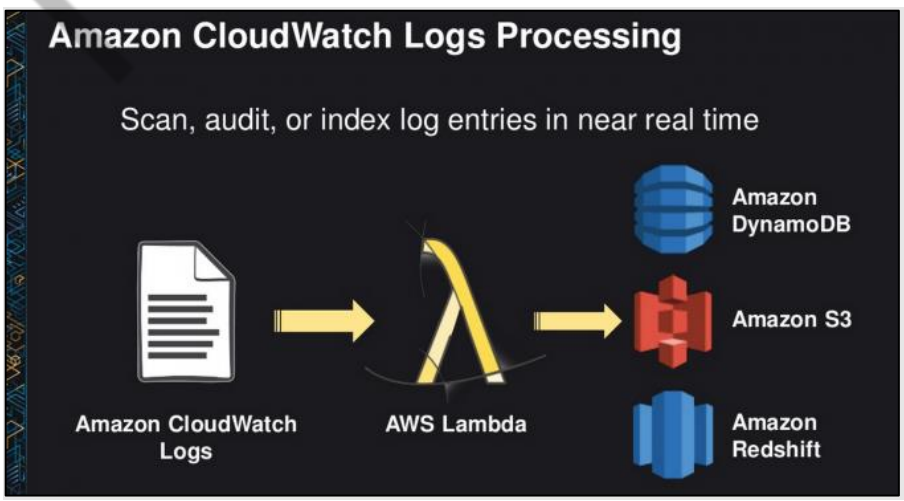

Figura 4 – Processamento de LOG
Fonte: Casalboni (2015)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 HANDS-ON – IMPLEMENTAÇÃO DE UMA FUNÇÃO SERVERLESS USANDO O PORTAL AZURE

O objetivo deste hands-on é implementar uma função simples (HelloWorld) disparada por uma chamada HTTP para mostrar as facilidades e agilidade de entregar um pequeno trecho de código funcional para Web sem necessidade de criação e configuração de servidores (AZURE, 2023).

Para implementar essa função, será necessário:

- Uma conta na nuvem Azure.

A seguir a sequência de ações que deve ser executada para disponibilizar uma função serverless:

Na plataforma Azure, escolher o serviço “Function App” (*Figura 5*).

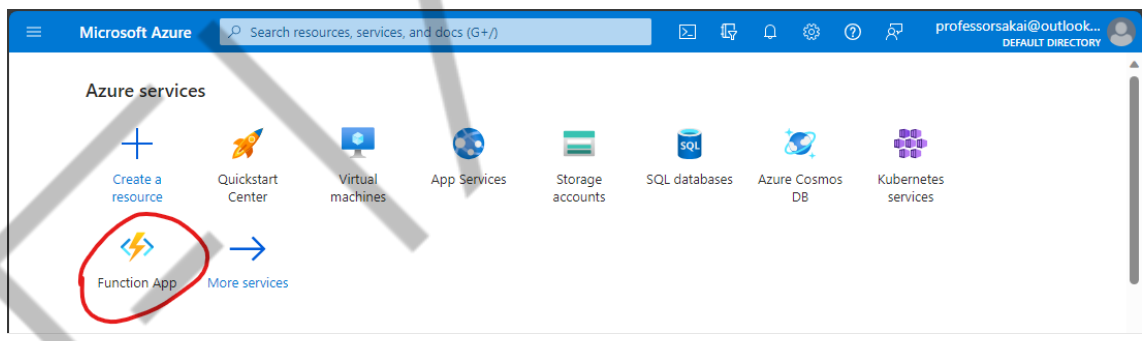

Figura 5 - Portal Azure - Function App
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na próxima tela, clicar em “+ Create” ou “Create Function App”, qualquer opção levará a mesma configuração.

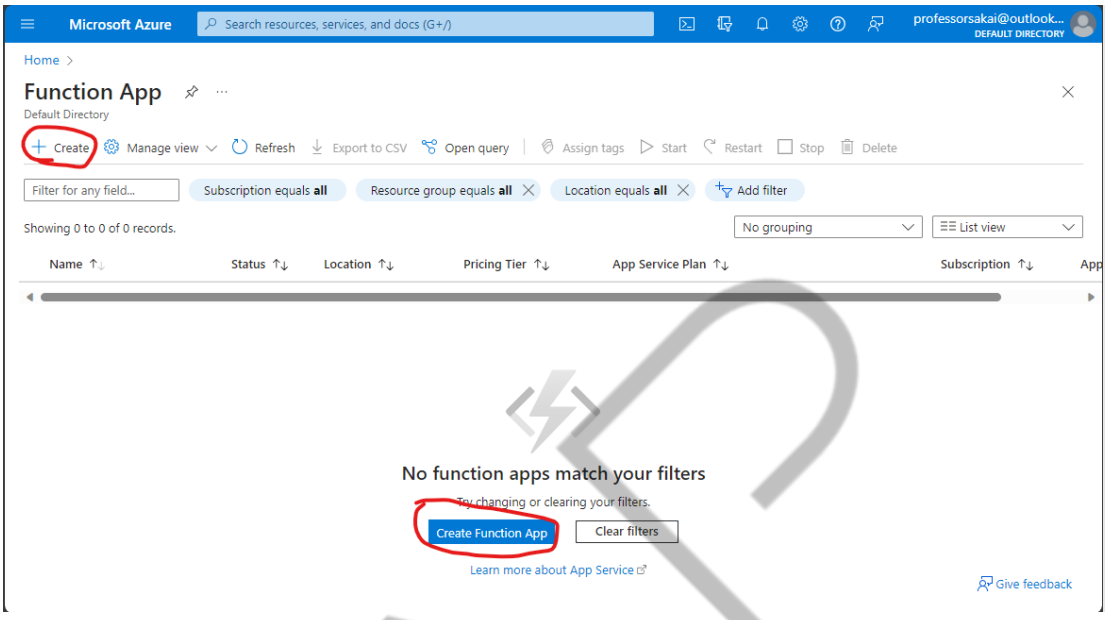

Figura 6 - Create Function
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A partir desse ponto, os detalhes de configuração da função devem ser preenchidos. As etapas a seguir devem ser seguidas pontualmente.

Configurações básicas, como conta, resource group, nome da função, são preenchidas nessa etapa.

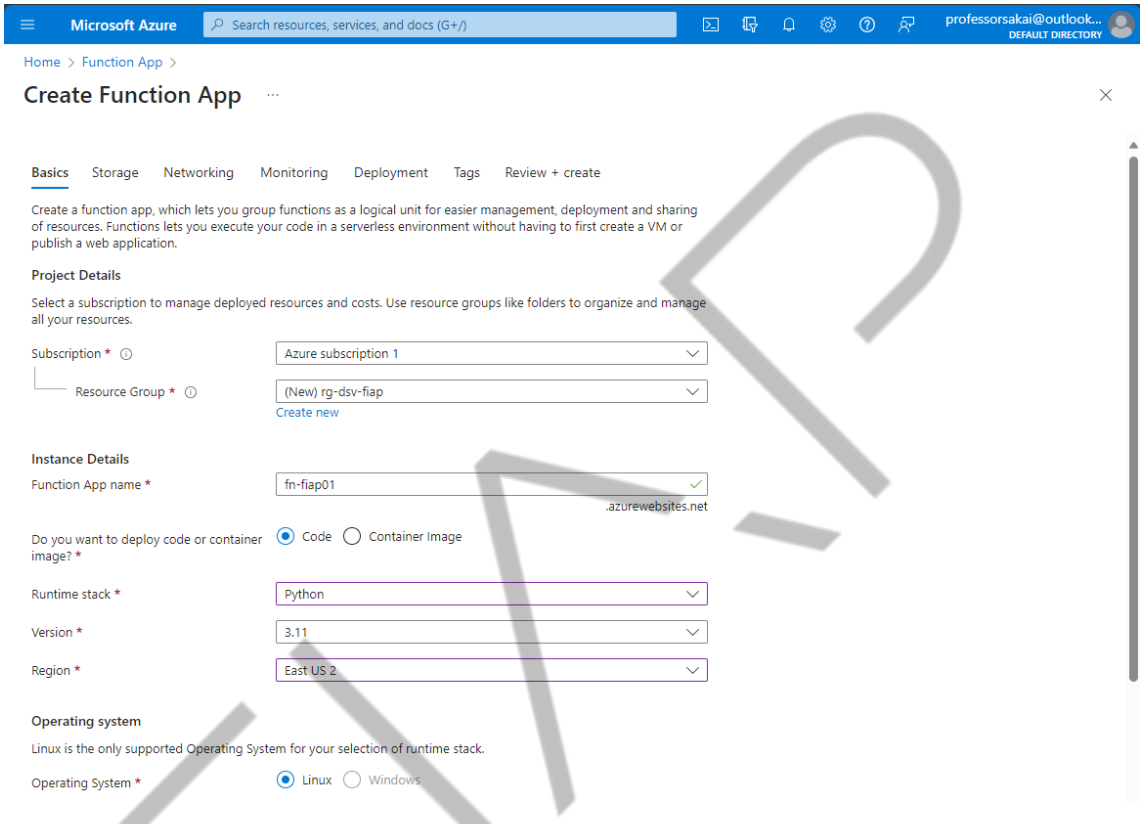

Figura 7 - Configurações Básicas
Fonte: Elaborado pelo autor (2023)

Subscription: Nome da conta Azure criada pelo aluno.

Resource Group: Pode ser um RG já existente ou criar um novo. Os resource groups (RGs) são estruturas lógicas para agrupar recursos em um mesmo local.

Function App name: nome da função que está sendo criada. O nome deve ser único em toda a nuvem Azure. Não pode haver um “function app” com o mesmo nome, mesmo que seja em outra conta de outra pessoa (ou empresa). Isso, porque o nome da função compõe a URL de chamada HTTP, que deve ser único na Internet.

Runtime Stack: Python

Version: a última versão estável disponível.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Region: East US 2 (pode escolher qualquer outra região que esteja disponível). A escolha das regiões não é aleatória, deve ser levado em conta itens como custo, recursos disponíveis, localização geográfica, etc.

Operating system: Linux (para a stack Python, somente o Linux está disponível).

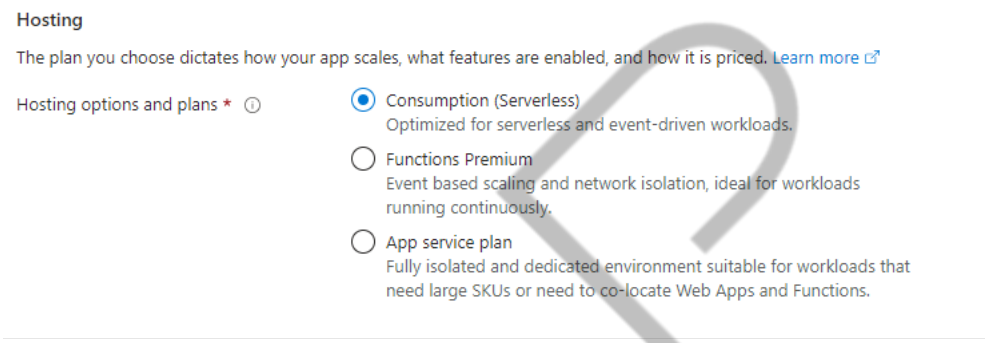

Figura 8 - Hosting - modo de consumo da Function App
Fonte: Elaborado pelo autor (2023)

O último conjunto de opções básicas determina a forma de cobrança e disponibilização da Function App. Aqui a opção sugerida para exemplificar o uso de Serverless é justamente “Consumption Serverless” (*Figura 8*).

Em Storage, manter o nome sugerido, será criado um novo storage para utilização da function app (*Figura 9*).

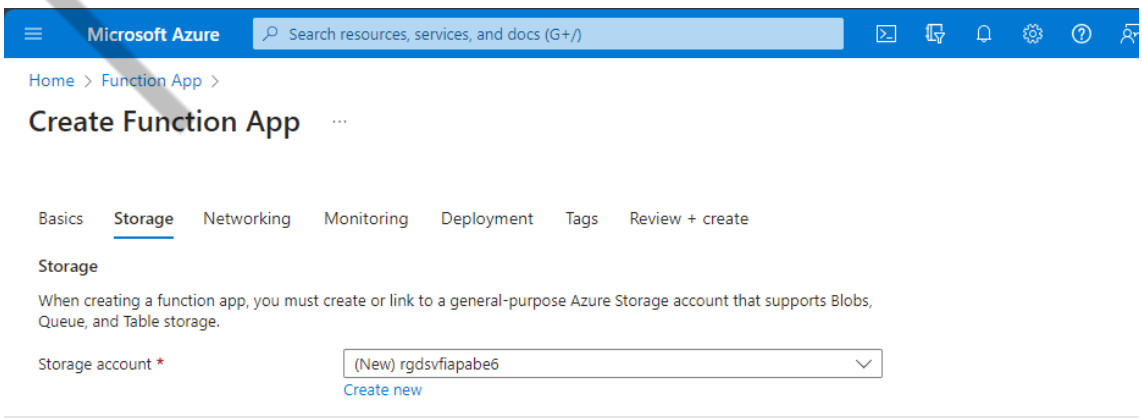

Figura 9 – Storage
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na configuração de rede, manter o acesso público habilitado. Servirá para utilização da function app à partir de qualquer local da internet (Figura 10).

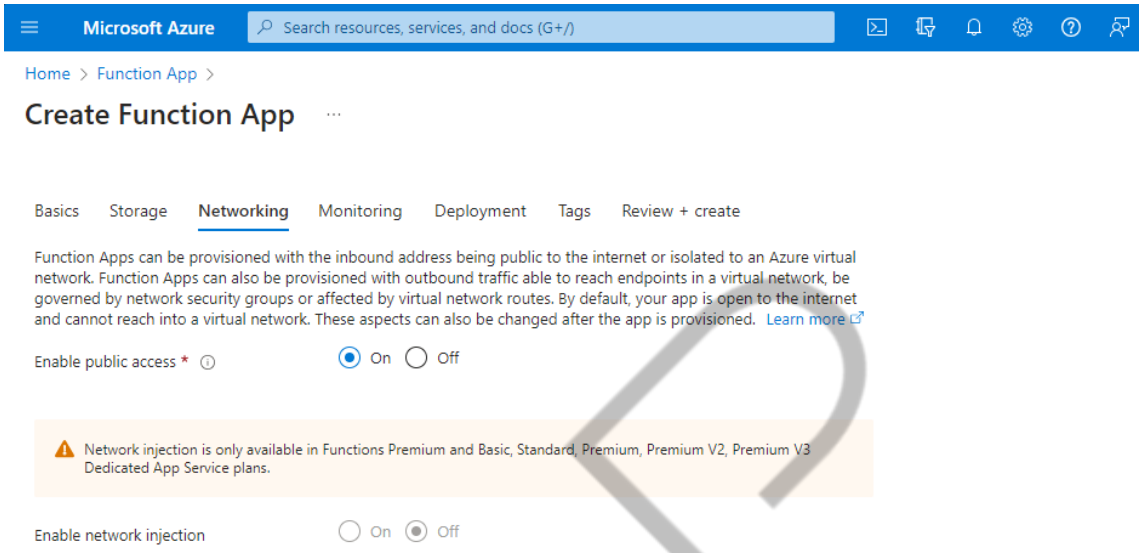

Figura 10 - Networking (rede)
Fonte: Elaborado pelo autor (2023)

Desabilitar a monitoração, por enquanto esse recurso não será utilizado (*Figura 11*).

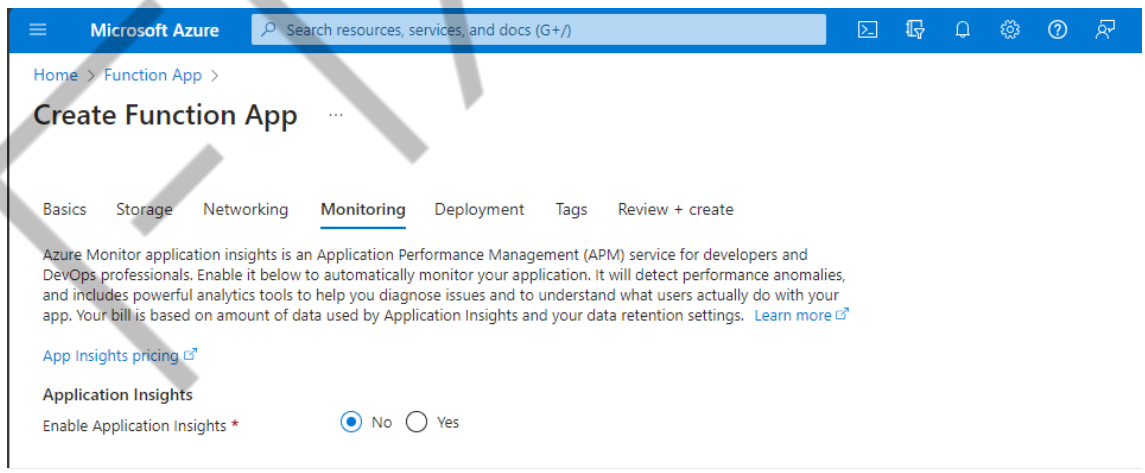

Figura 11 – Monitoring
Fonte: Elaborado pelo autor (2023)

A forma de entrega (deploy) pode ser automática, usando o GitHub para fazer o gerenciamento das modificações, compilação e entrega da aplicação. No momento será configurado como desabilitado (*Figura 12*). O código da aplicação será criado e entregue de forma manual.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

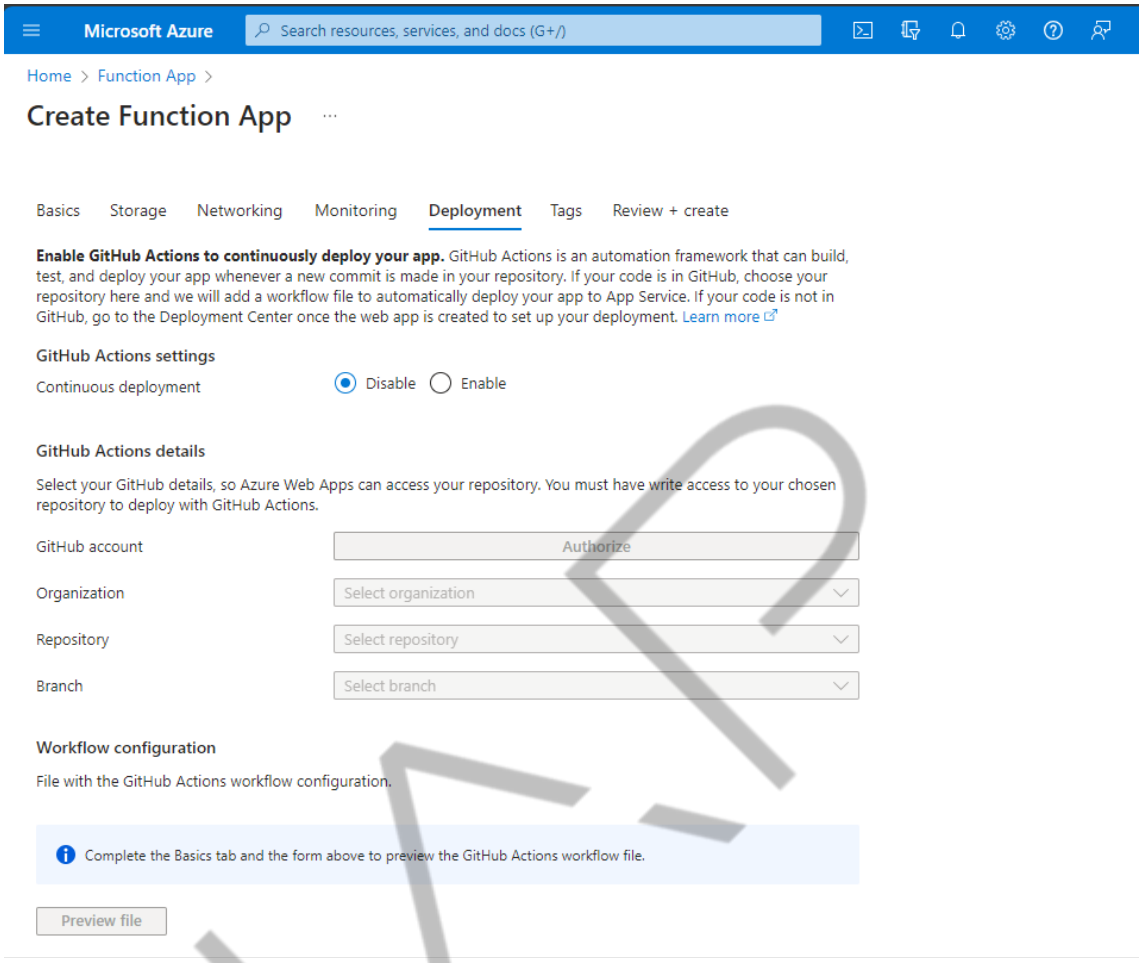

Figura 12 - Deployment
Fonte: Elaborado pelo autor (2023)

As TAGs devem ser preenchidas conforme necessidade de classificação da utilização do recurso. Muito utilizado pela equipe de controle financeiro da conta Azure. A figura abaixo, mostra o exemplo de duas TAGs: ops-ambiente e ops-produto.

Clicar em “Review + create”, caso todas as configurações obrigatórias estejam corretas, o botão de “Create” se tornará disponível (*Figura 13*). Caso contrário, será mostrado o problema a ser corrigido.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

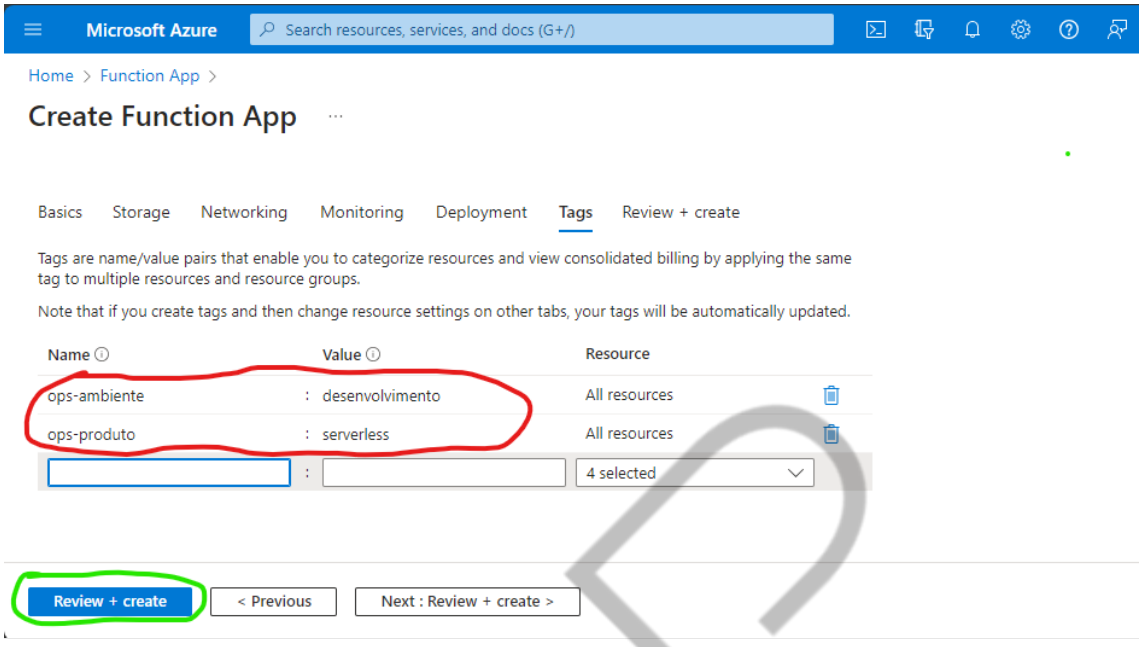

Figura 13 – Configuração de TAGs
Fonte: Elaborado pelo autor (2023)

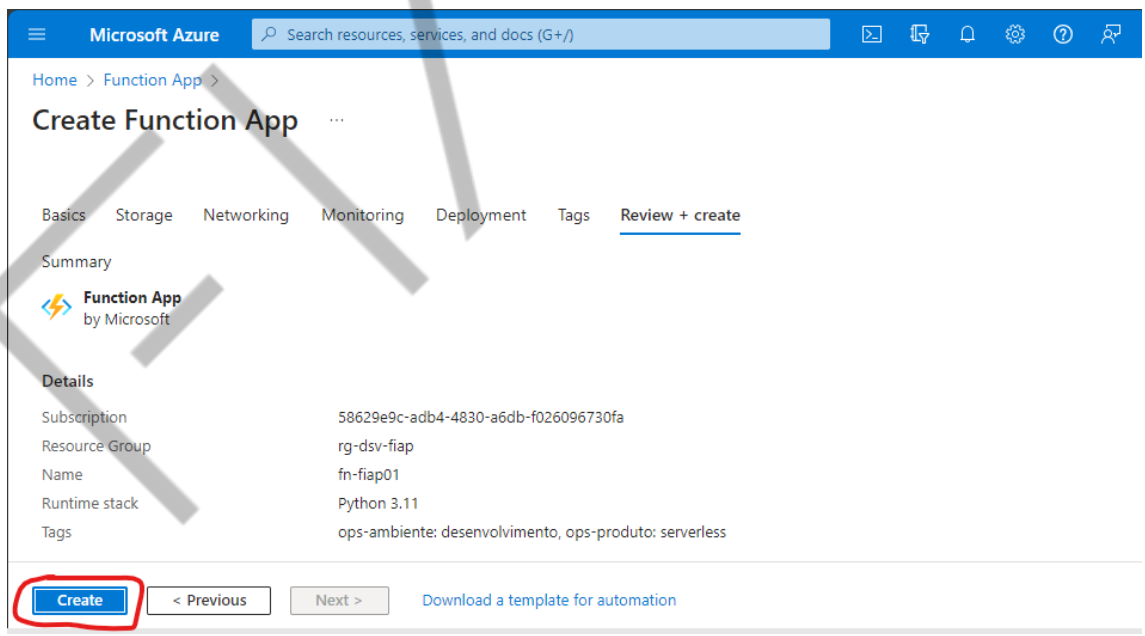

Figura 14 - Revisar e criar
Fonte: Elaborado pelo autor (2023)

Assim que o recurso estiver disponível, será apresentada uma tela conforme *Figura 15*. Clicar em “Go to resource” para continuar a criação da função serverless”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

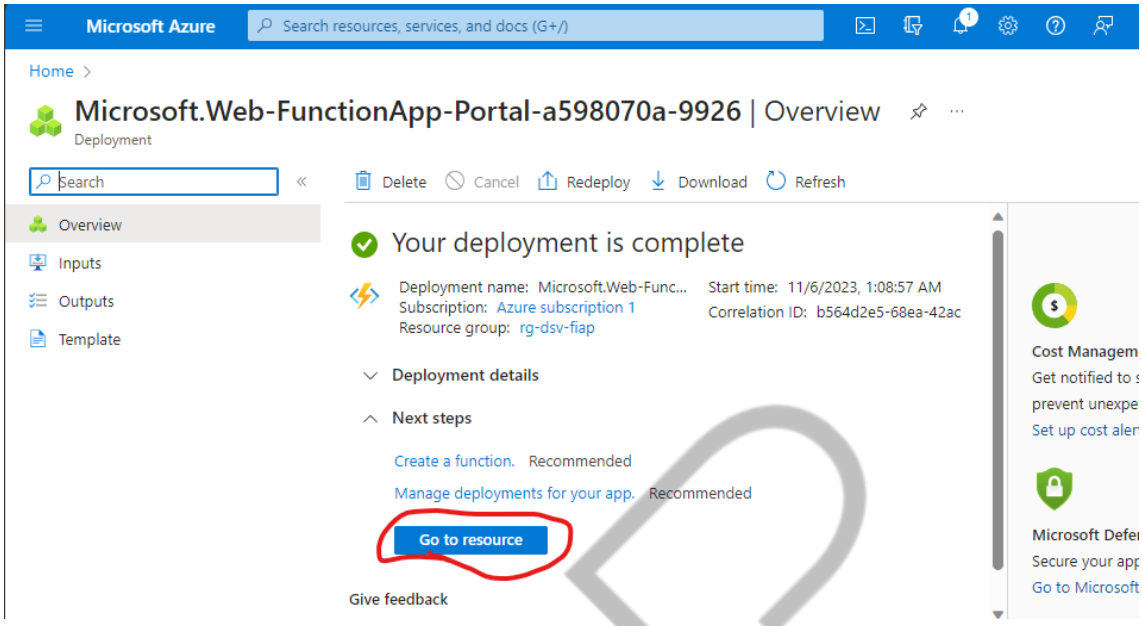

Figura 15 - Disponibilização da Function App
Fonte: Elaborado pelo autor (2023)

Criar uma função simples

Na área principal da “Function App” criada, clicar no botão no meio da página “Create in Azure portal”, conforme *Figura 16*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

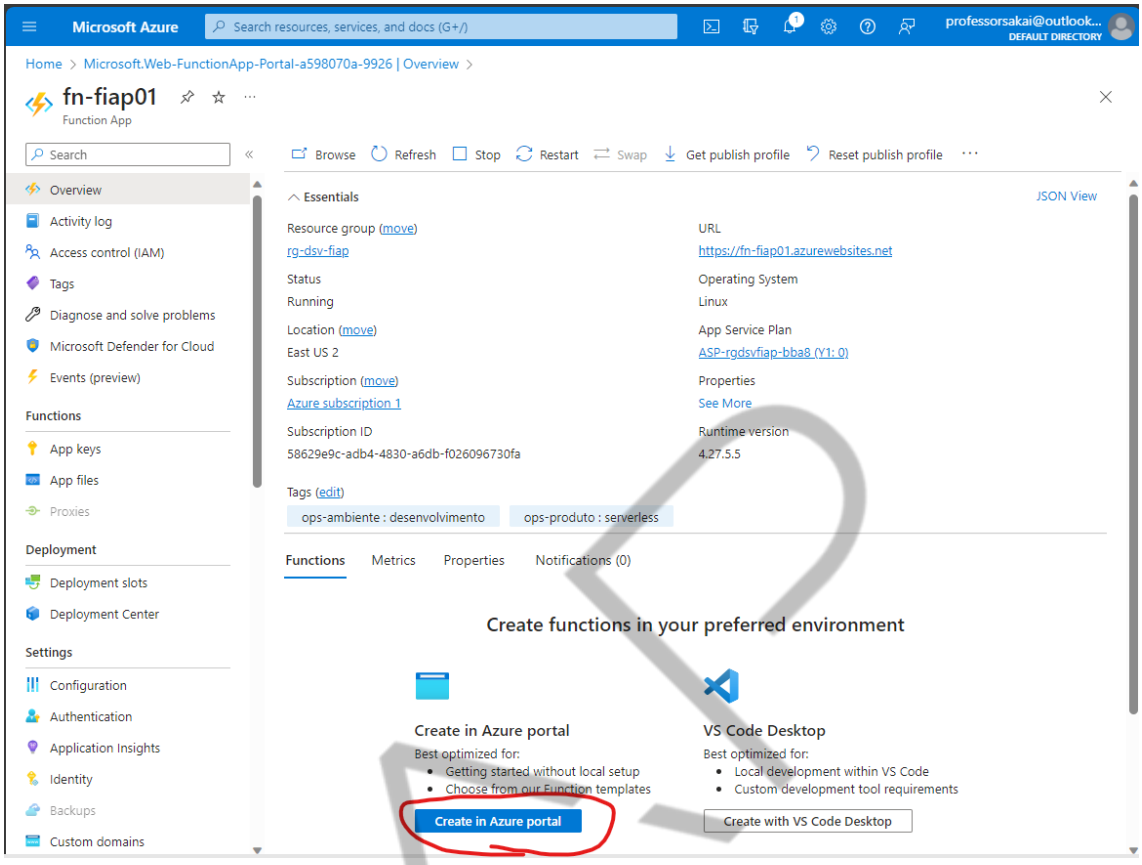

Figura 16 - Criar uma função
Fonte: Elaborado pelo autor (2023)

As funções serverless devem ser disparadas por algum gatilho externo, uma chamada HTTP, um evento de CosmoDB, um evento de HUB entre outros. Nessa etapa será criada uma simples função que pode ser disparada por uma chamada HTTPS utilizando o endereço da função.

Manter a configurações conforme padrão e escolher “HTTP trigger” (*Figura 17*).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

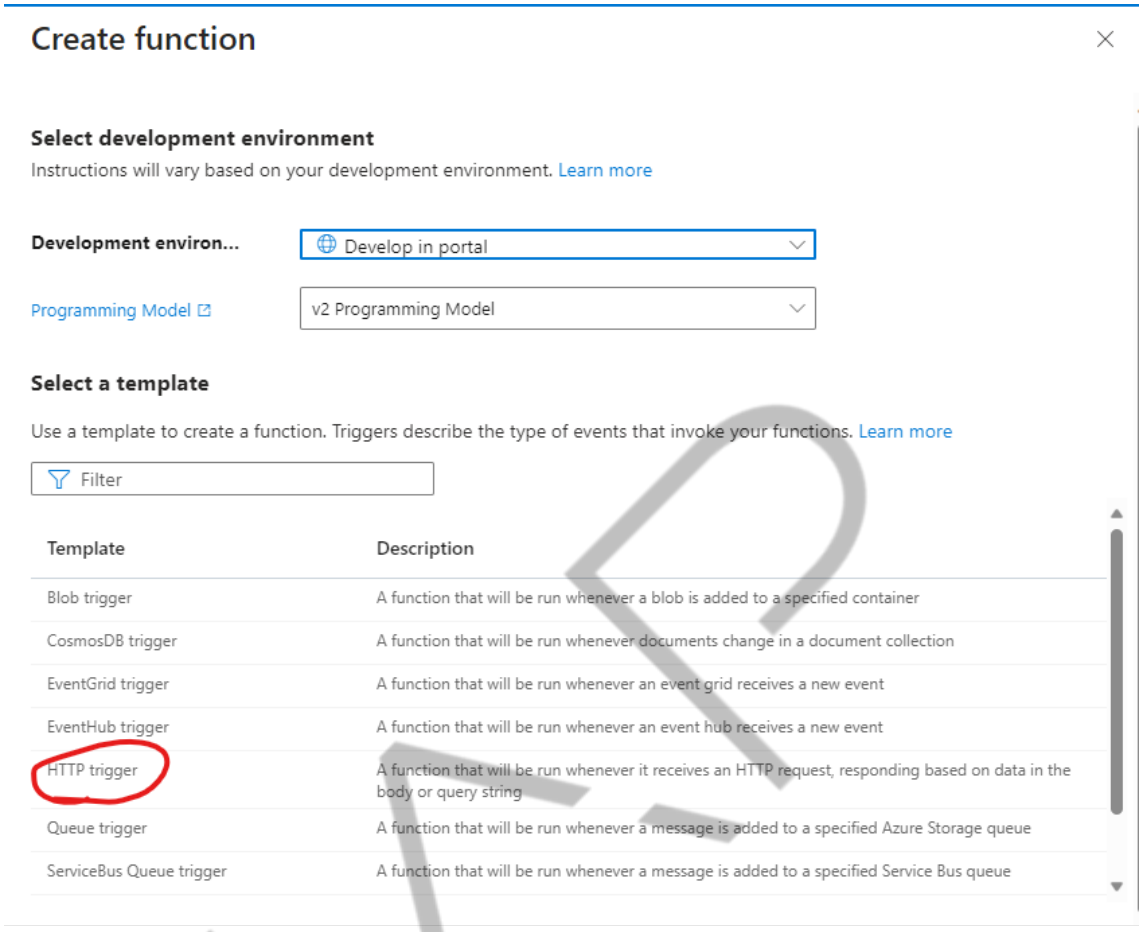

Figura 17 - Criação de uma função HTTP Trigger
Fonte: Elaborado pelo autor (2023)

Serão abertas as opções conforme *Figura 18*. Preencher as informações e clicar em “Create”. Nesse passo, observar principalmente o “Authorization Level”, pois determina a forma de autenticação necessária para chamar a função. Manter como anônimo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

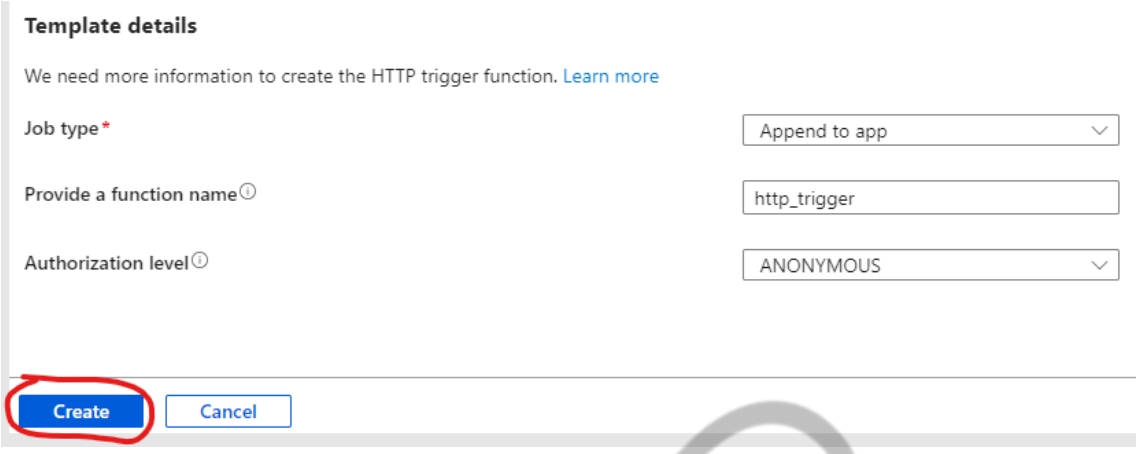

Figura 18 - Configuração da função HTTP Trigger
Fonte: Elaborado pelo autor (2023)

Codificar a função criada.

Manter o exemplo gerado automaticamente na criação da função. Nesse momento, o objetivo é apresentar a facilidade e rapidez de desenvolver e entregar uma função serverless. A *Figura 19* mostra o código gerado automaticamente.

Figura 19 - Código gerado automaticamente
Fonte: Elaborado pelo autor (2023)

Para testar a função, copie a URL conforme mostrado na *Figura 20* e cole em uma outra aba do navegador.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

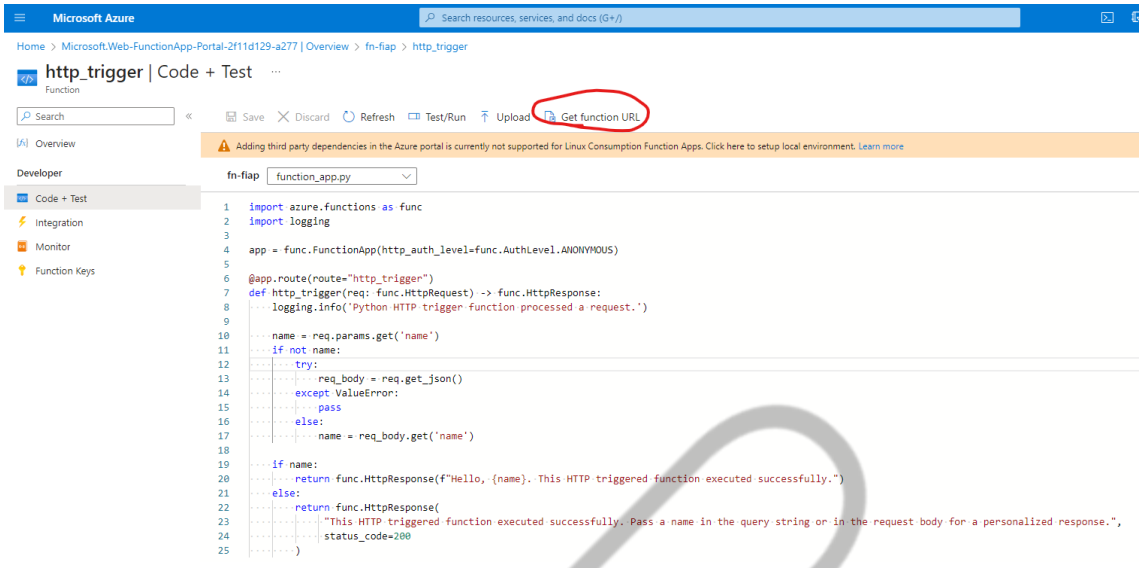

Figura 20 - URL da função gerada
Fonte: Elaborado pelo autor (2023)

Após a primeira execução, a página abaixo será apresentada (Figura 21).

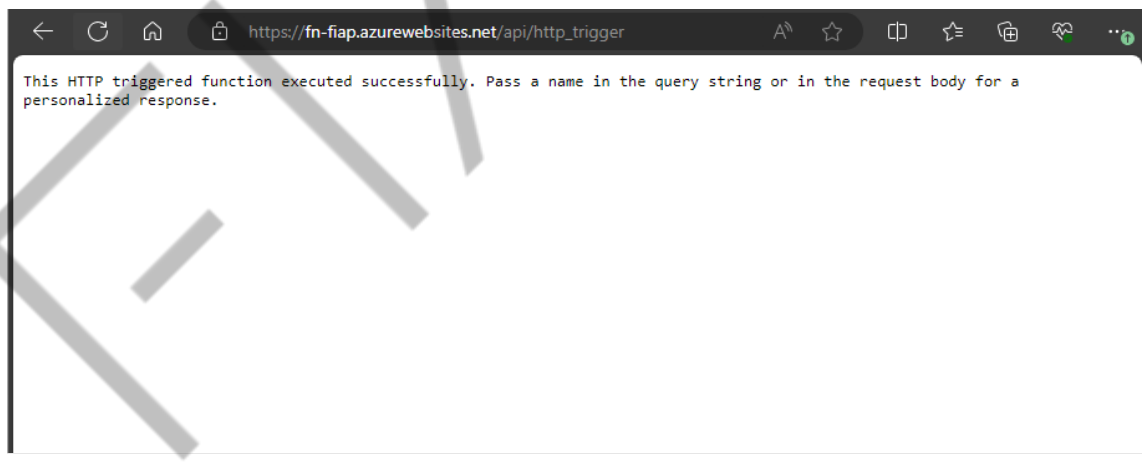

Figura 21 - Teste da função
Fonte: Elaborado pelo autor (2023)

Agora, adicionando ao final da URL a seguinte string “?name=Rodrigo” (coloque seu nome). Veja a modificação da mensagem que ocorre na página (*Figura 22*).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

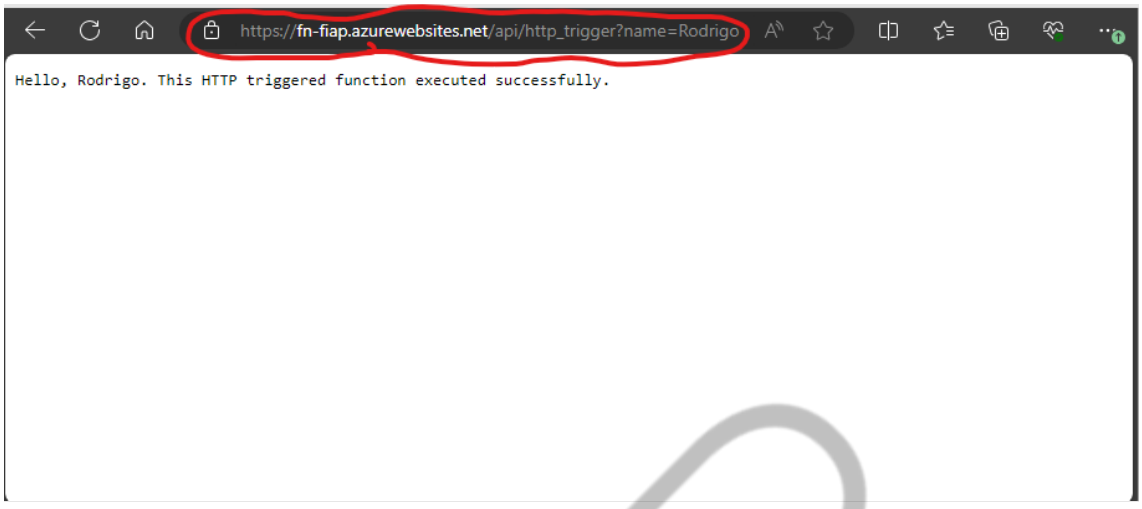

Figura 22 - Teste da função
Fonte: Elaborado pelo autor (2023)

No exemplo, foram realizados alguns passos para a entrega de uma aplicação simples (HelloWorld). Porém, as empresas têm usado cada vez mais a arquitetura *serverless* e outros recursos na nuvem para disponibilizar desde serviços pontuais, como previsão do tempo, localização geográfica ou processamento de imagens para web, até sistemas completos, como gerenciamento de produtos, estoques e empregados.

É possível observar que não foi necessário instalar e configurar um servidor com sistema operacional, compiladores, bibliotecas de funções, serviços de web, etc. Assim, a programação de aplicações torna-se mais produtiva.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ATIVIDADE TREINO

Esta atividade não é avaliativa, caso queira ter um feedback, entre em contato com seu tutor pelo Teams.

1. A arquitetura *serverless* traz alguns benefícios para os desenvolvedores, por exemplo, a independência da infraestrutura para disponibilizar as funções desenvolvidas. Como isso pode melhorar a qualidade dos códigos?

2. As principais infraestruturas de nuvem (Microsoft Azure, AWS, Oracle Cloud, Google Cloud) oferecem o serviço de computação *serverless*. Qual o nome desses serviços no AWS e no Microsoft Azure?

3. A arquitetura desacoplada é um tipo de arquitetura de computação que permite que componentes ou partes do sistema sejam executados independentemente, porém ainda fazem interface uns com os outros. Qual a principal diferença entre a arquitetura monolítica e a desacoplada?

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

AWS. **AWS Lambda:** execute código sem se preocupar com servidores ou clusters. 2022. Disponível em: https://aws.amazon.com/pt/lambda/. Acesso em: 15 jan. 2022.

CASALBONI, A. **AWS Lambda and the Serverless Cloud**. 2015. Disponível em: https://cloudacademy.com/blog/aws-lambda-serverless-cloud/. Acesso em: 20 jan. 2022.

CLOUDFLARE. **O que é computação sem servidor?** 2022. Disponível em: https://www.cloudflare.com/pt-br/learning/serverless/what-is-serverless/. Acesso em: 28 dez. 2021.

DOMALIK, V. **Lost in Serverless**. 2019. Disponível em: https://blogs.oracle.com/cloudnative/post/lost-in-serverless. Acesso em: 27 dez. 2021.

IBM. **Serverless**. 2021. Disponível em: https://www.ibm.com/cloud/learn/serverless. Acesso em: 27 dez. 2021.

AZURE. **Criar sua primeira função no portal do Azure**. 2023. Disponível em: Criar sua primeira função no portal do Azure | Microsoft Learn. Acesso em: 05 dez. 2023.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com